Anime Recommendation System: A Comprehensive Report

Petros Triantafyllos

February 12, 2025

Abstract

This report presents the development and evaluation of an anime recommendation system. The work utilizes collaborative filtering methods with bias estimation and truncated Singular Value Decomposition (SVD) to extract latent factors, followed by an exploration of item- and user-based cosine similarity in the latent space. Despite attempts to use cosine similarity on its own and in combination with SVD, these methods did not yield satisfactory results. Truncated SVD was chosen primarily for its efficiency with only a small relative loss in accuracy. Early in the project, a chronological split for evaluation was considered, but it was abandoned due to issues such as bias introduced by users who rate only new shows. In addition, this report discusses the role of hyperparameter λ for regularization. Experimental results, evaluation metrics, and proposed extensions are also presented.

1 Introduction

Recommender systems have become indispensable in guiding users through the vast content available on digital platforms. In this project, an anime recommendation system was developed using collaborative filtering methods. The approach combines bias estimation with truncated Singular Value Decomposition (SVD) to capture latent user–item interactions. In an effort to further refine recommendations, cosine similarity was applied both on the latent factors generated by SVD and directly on the rating data; however, these methods did not produce significant improvements. This report outlines the data sources, methodology, evaluation, and conclusions of the project.

2 Data Source

The primary data for this project was obtained from a Kaggle Anime dataset, which contains extensive metadata for thousands of anime, including the title, genre, type, number of episodes, overall rating, membership counts, airing information, and the number of reviews (the "Scored By" column). This dataset was enhanced using the MyAnimeList Jikan API, which provided additional fields such as precise airing dates. Alongside the anime metadata,

a separate ratings dataset was used, which records individual user ratings for each anime. In the ratings dataset, missing ratings (represented by a value of -1) were removed. Both datasets were merged on the common key `anime_id` to obtain a comprehensive view of both the item properties and user feedback.

3 Data Exploration

Two main datasets were explored: the anime dataset and the ratings dataset. The anime dataset originally comprised over 10,000 entries, but after filtering out shows with fewer than 1000 reviews (as indicated by the `Scored By` field), the dataset was reduced to 6,479 shows. After merging, the combined dataset contains over 5.6 million rating events. It is important to note that the merged dataset's large size reflects the total number of rating events rather than the number of unique anime or users. For instance, while there are approximately 6,400 unique anime and around 70,000 unique users, many popular anime receive thousands of ratings.

Visualizations were generated to illustrate key distributions:

- A histogram of overall anime ratings to show the distribution of ratings.
- A log-scale histogram of the number of ratings per anime, highlighting the heavy-tailed nature of popularity.
- A log-scale histogram of the number of ratings per user, showing the diversity in user activity.

These figures provide insight into the sparsity and distribution of ratings, which informs subsequent modeling decisions.

4 Algorithm and Hyperparameters

The core model of the recommendation system is based on collaborative filtering with bias estimation and truncated SVD. The key steps are as follows:

Bias Estimation and Residual Modeling

The rating matrix R is built from the merged dataset, where each entry $r_{u,i}$ represents the rating given by user u to anime i. First, the global mean μ of all observed ratings is computed. Then, user biases b_u and item biases b_i are estimated iteratively using regularized updates:

$$b_i \leftarrow \frac{\sum_{u \in U(i)} (r_{u,i} - \mu - b_u)}{\lambda + |U(i)|}, \quad b_u \leftarrow \frac{\sum_{i \in I(u)} (r_{u,i} - \mu - b_i)}{\lambda + |I(u)|},$$

where λ is the regularization parameter. In our experiments, λ was tuned via **grid search** (and Bayesian optimization in some experiments) and was finally set to 20. This hyperparameter prevents overfitting by shrinking bias estimatess.

After the biases are estimated, the residual matrix is computed as:

$$R_{\text{res}} = R - \mu - b_u - b_i.$$

This residual matrix is then factorized using truncated SVD, which decomposes R_{res} into low-dimensional latent factors. Truncated SVD was chosen for its efficiency; it approximates the original matrix with a small relative loss in accuracy while significantly reducing computational complexity.

Cosine Similarity Approaches

An attempt was made to incorporate cosine similarity on the latent factors extracted from SVD and directly on the rating matrix for both item- and user-based collaborative filtering. In the latent space, cosine similarity was computed between item vectors (or user vectors), with the hope of re-ranking recommendations based on similarity measures. However, these approaches did not yield improved performance. The main model remains the SVD-based collaborative filtering with bias adjustment.

Abandoned Chronological Split

Initially, the project aimed to split the data chronologically to simulate a real-time recommendation scenario. However, this approach was abandoned due to several issues: for example some users only rate new shows, which in a chronological fold/ test set would introduce a bias that complicates evaluation. Instead, a leave-10-out cross-validation strategy was adopted, ensuring that every user contributes to both training and test sets without the temporal bias inherent in a chronological split.

5 Testing and Evaluation

For evaluation, a leave-10-out strategy was used. For each user, 10 ratings were randomly held out from the dataset as test data, and the remaining ratings were used for training. The model's performance was evaluated in terms of Precision@10 and Recall@10. These metrics are computed as follows:

$$\text{Precision@10} = \frac{\text{Number of relevant items in top 10 recommendations}}{10}$$

$$\text{Recall@10} = \frac{\text{Number of relevant items in top 10 recommendations}}{\text{Total number of relevant items for the user}}.$$

An item was considered "relevant" if the rating was greater than or equal to 8.

In addition, to analyze the ranking performance further, ROC and Precision-Recall curves were generated. These curves were computed by thresholding the predicted scores to produce binary labels (relevant vs. non-relevant) and comparing them with the actual labels.

6 Results

After training and evaluating the model, the following results were obtained:

- The merged dataset, after filtering shows with less than 1000 reviews and users with fewer than 50 reviews, contained approximately 5.66 million rating records.
- The leave-10-out split produced a training set with 5,331,638 records and a test set with 329,390 records.
- The truncated SVD step completed in approximately 7.21 seconds.
- The average Precision@10 was approximately 0.021 and the average Recall@10 was approximately 0.017.
- The ROC AUC was around 0.65.

These results indicate that while the model is able to capture some relevant signals from the data, the absolute values of the precision and recall remain quite low—an issue common in large, sparse recommendation systems. It is likely that further improvements, such as hybrid approaches or additional feature integration, are needed to boost performance.

7 Proposed Extensions and Future Work

Future work will focus on incorporating content-based features to mitigate the cold-start problem and to improve overall accuracy. In particular, combining the collaborative filtering signals with content information (such as genre, synopsis, and other metadata) is a promising direction. One approach involves using embeddings derived from large language models (LLMs) to capture semantic information from textual descriptions, which can then be combined with the latent factors from SVD.

Although attempts were made to integrate cosine similarity (both in the latent space and directly on rating vectors) into the recommendation process, these methods did not provide significant improvements. Further investigation into blending similarity-based signals with the SVD-based predictions might yield better performance.

8 Conclusion

This report detailed the development of an anime recommendation system based on collaborative filtering, using bias estimation and truncated SVD. The choice of truncated SVD was driven primarily by the efficiency gains it provides with only a minimal loss in accuracy. The hyperparameter λ plays a crucial role in regularizing the bias estimates, and our experiments showed that a value of 20 offered a reasonable balance. Although a chronological split was considered initially, it was ultimately abandoned due to the potential introduction of bias. Instead, a leave-10-out strategy was employed to ensure robust evaluation.

The final evaluation results indicate that while the model is able to capture some relevant signals (with an average Precision@10 of around 2.1% and Recall@10 of around 1.7%), there remains substantial room for improvement. Future enhancements, particularly hybrid approaches that combine collaborative filtering with content-based methods, appear promising.

9 Appendix: Figures and Diagrams

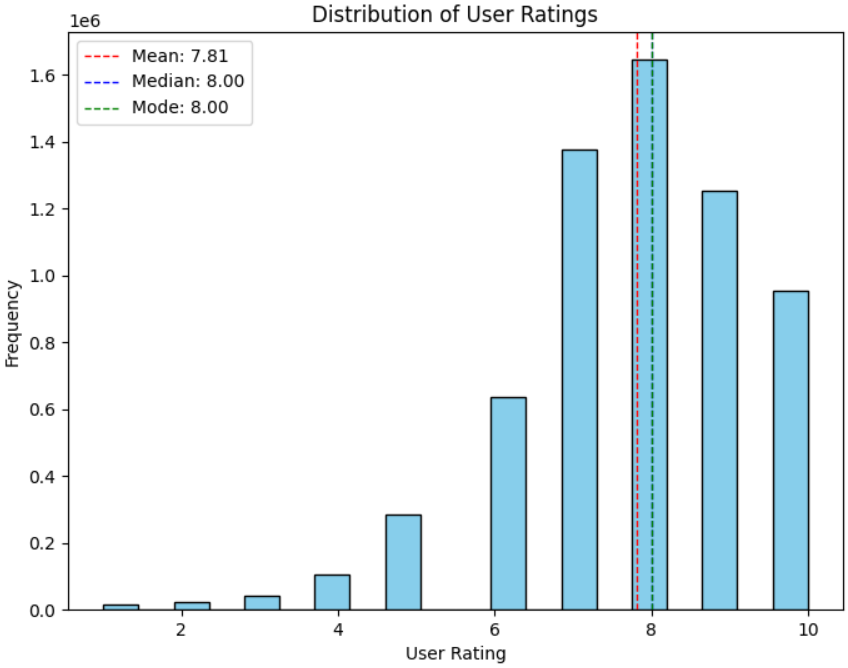

Figure 1: Distribution of Overall Anime Ratings.

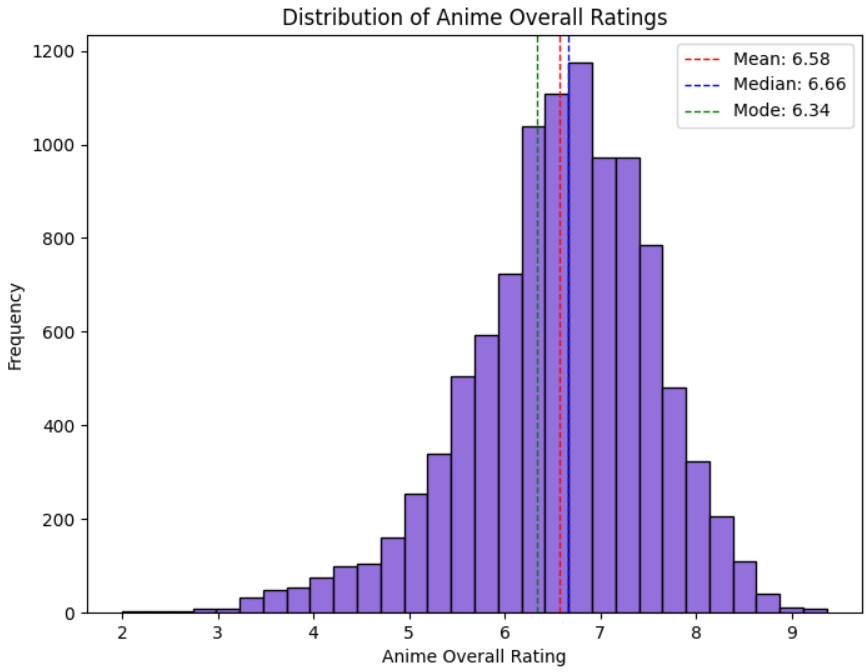

Figure 2: Distribution of User Ratings.

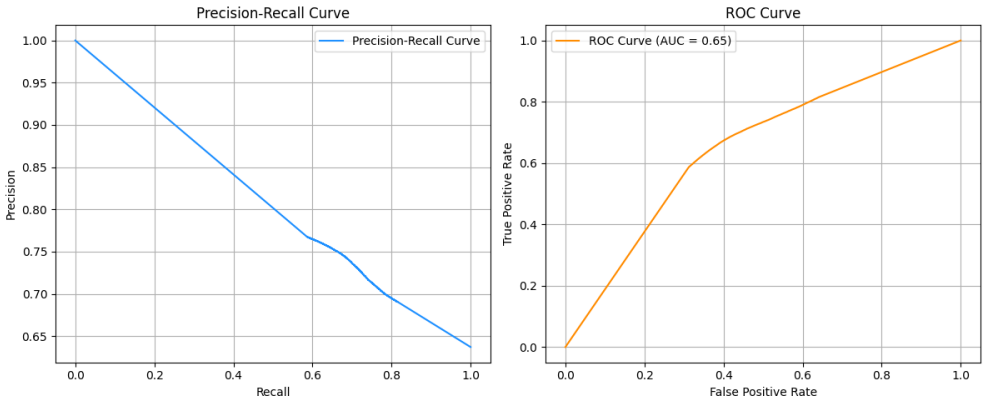

Figure 3: Precision-Recall and ROC Curves.